Campus Mind: AI College Assistance Chatbot

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Lokesh Phund – EN23CS301567, Kunal Mahajan – EN23CS301543,
Lakshay Dekhwar – EN23CS301560

Under the Guidance of

Dr. Madan Pal Singh , Dr. Mahesh Patidar

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE - 453331

APRIL-2026

Report Approval

The project work **"Campus Mind: AI College Assistance Chatbot"** is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal
Examiner
Name:
Designation
Affiliation

External
Examiner Name:
Designation
Affiliation

Declaration

I/We hereby declare that the project entitled **"Campus Mind: AI College Assistance Chatbot"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications in 'Computer Science and Engineering' completed under the supervision of **Dr. Madan Pal Singh , Dr. Mahesh Patidar and Computer Science and Engineering ,** Faculty of Engineering , Medicaps University Indore is an authentic work.

Further, I/we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Lokesh Phund[EN23CS301567]

Kunal Mahajan[EN23CS301543]

Lakshay Dekhwar[EN23CS301560]

Signature and name of the student(s) with date

Certificate

I/We, **Dr. Madan Pal Singh , Dr. Mahesh Patidar** certify that the project entitled **"Campus Mind: AI College Assistance Chatbot"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications by **Lokesh Phund – EN23CS301567 , Kunal Mahajan – EN23CS301543 , Lakshay Dekhwar – EN23CS301560** is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

_____

Name of Internal Guide

Dr. Madan Pal Singh

Dr. Mahesh Patidar

Computer Science & Engineering

MediCaps University, Indore

_____

Prof. (Dr.) Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal,** who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik,** Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. Ratnesh Itoriya,** Dean, Faculty of Engineering, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Prof. (Dr.) Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Lokesh Phund [EN23CS301567]

Kunal Mahajan [EN23CS301543]

Lakshay Dekhwar [EN23CS301560]

B.Tech. III Year (J)

Department of Computer Science & Engineering

Faculty of Engineering

Medicaps University, Indore

Abstract

"Campus Mind" is an AI-powered college assistance chatbot designed to simplify and automate student queries related to academic and administrative information. Traditional methods of obtaining college-related information are often time-consuming and inefficient. This system provides an intelligent, real-time conversational interface that allows students to interact with a chatbot and receive accurate responses instantly.

The chatbot is developed using Python and Flask for backend processing, while the frontend is built using HTML, CSS, and JavaScript to provide a clean and responsive user interface. The system leverages Natural Language Processing (NLP) techniques using the NLTK library for understanding user input. A custom implementation of TF-IDF and Cosine Similarity is used to match user queries with the most relevant responses stored in a JSON-based knowledge base.

The chatbot processes user queries by tokenizing input text, removing stopwords, and applying stemming techniques. It then computes similarity scores between user input and stored patterns to generate appropriate responses. This lightweight and efficient system eliminates the need for complex database management by using a structured JSON file.

Campus Mind enhances accessibility, reduces workload on administrative staff, and provides students with quick and reliable information, making it a practical solution for modern educational institutions.

Keywords:

- Artificial Intelligence (AI)
- Chatbot
- Natural Language Processing (NLP)
- TF-IDF
- Cosine Similarity
- Flask Framework
- JSON Data Handling

- College Assistance System
- Web-based Application
- Student Query Automation

Table of Contents

List of Figures

List of Tables

Abbreviations

- AI – Artificial Intelligence
- NLP – Natural Language Processing
- UI – User Interface
- API – Application Programming Interface
- TF-IDF – Term Frequency-Inverse Document Frequency
- JSON – JavaScript Object Notation
- HTML – HyperText Markup Language
- CSS – Cascading Style Sheets
- JS – JavaScript
- DB – Database
- HTTP – HyperText Transfer Protocol
- ML – Machine Learning

Notations & Symbols

- TF– Term Frequency (frequency of a word in a document)
- IDF– Inverse Document Frequency (importance of a word)
- $TF\text{-}IDF$– Product of TF and IDF used for vectorization
- θ– Similarity threshold value
- x– Input query vector
- y– Stored data vector
- $sim(x, y)$– Cosine similarity between vectors
- Σ– Summation symbol used in calculations
- n– Total number of documents
- d– Document (text data unit

Chapter-1(Introduction)

1.1 Introduction

In the modern digital era, educational institutions are increasingly adopting advanced technologies to enhance communication and efficiency. Students frequently require information related to academic schedules, examination details, admission procedures, fee structures, and other administrative queries. Traditionally, such information is obtained by visiting college offices or searching through websites, which is often time-consuming and inefficient.

To overcome these challenges, an intelligent solution in the form of an AI-based chatbot can be implemented. “Campus Mind” is an AI-powered college assistance chatbot designed to provide instant responses to student queries through a conversational interface. The system enables users to interact in natural language, making it user-friendly and accessible.

The chatbot is built using modern web technologies such as HTML, CSS, and JavaScript for the frontend, and Python with Flask for backend processing. It utilizes Natural Language Processing (NLP) techniques to understand and process user queries effectively. By leveraging a custom implementation of TF-IDF and Cosine Similarity, the chatbot matches user input with predefined responses stored in a JSON-based knowledge base.

The system aims to reduce dependency on manual support systems, improve accessibility, and provide a seamless experience to students by delivering quick and accurate information.

1.2 Literature Review

Artificial Intelligence and Natural Language Processing have significantly evolved in recent years, enabling the development of intelligent chatbot systems. Early chatbots were primarily rule-based systems that relied on predefined patterns and keyword matching.

These systems were limited in understanding user intent and often failed to handle complex queries.

Modern chatbot systems incorporate NLP techniques such as tokenization, stopword removal, and stemming to preprocess user input. Machine learning approaches have further enhanced chatbot capabilities by allowing systems to learn from data and improve over time.

Several research studies highlight the effectiveness of TF-IDF and Cosine Similarity in text-based information retrieval systems. TF-IDF helps in identifying the importance of words within a document, while Cosine Similarity measures the similarity between two text vectors. These techniques are widely used in lightweight chatbot systems where computational efficiency is important.

In educational environments, chatbots are increasingly used to automate student support services. They help in answering frequently asked questions, reducing workload on administrative staff, and improving response time. However, many existing systems rely on heavy machine learning models or require large datasets, making them complex and resource-intensive.

The Campus Mind chatbot addresses these limitations by implementing a lightweight and efficient NLP-based approach using custom-built algorithms, ensuring both performance and simplicity.

1.3 Objectives

The primary objectives of the Campus Mind chatbot are:

- To develop an AI-based chatbot for handling college-related queries
- To provide real-time and accurate responses to users
- To implement Natural Language Processing techniques for understanding user input
- To design a simple and user-friendly interface
- To reduce manual workload on college staff

- To build a lightweight system using JSON-based data storage

---

1.4 Significance of the Study

The Campus Mind chatbot plays a significant role in improving communication within educational institutions. It provides a centralized platform where students can obtain information instantly without relying on manual processes.

The system enhances efficiency by reducing the time required to access information. It also minimizes the workload on administrative staff by automating repetitive queries. Additionally, the chatbot ensures 24/7 availability, allowing students to access information anytime and from anywhere.

From a technical perspective, the project demonstrates the practical implementation of NLP techniques such as TF-IDF and Cosine Similarity. It also highlights the effectiveness of lightweight AI solutions in real-world applications.

1.5 Research Design

The project follows an experimental and implementation-based research approach. The system is designed, developed, and tested to evaluate its effectiveness in handling user queries.

Methodology:

Phase 1: Data Collection

- Creation of dataset in JSON format
- Defining intents, patterns, and responses

Phase 2: Text Preprocessing

- Tokenization of user input
- Removal of stopwords
- Stemming using Porter Stemmer

Phase 3: Feature Extraction

- Conversion of text into numerical form using TF-IDF

Phase 4: Similarity Matching

- Calculation of Cosine Similarity between user query and stored patterns

Phase 5: Response Generation

- Selection of best matching response
- Display output to user

1.6 Source of Data

The data used in this project is stored in a JSON file named **data.json**. It contains structured information in the form of:

- Intents (categories of queries)
- Patterns (possible user inputs)
- Responses (predefined replies)

The dataset is manually created and designed to cover common college-related queries such as admissions, courses, exams, and general information.

1.7 Chapter Scheme

The project report is organized into multiple chapters for clear understanding:

- **Chapter 1**: Introduction – Overview of the project, objectives, and significance
- **Chapter 2**: Requirement Specification – Functional and non-functional requirements
- **Chapter 3**: Design – System architecture and algorithm
- **Chapter 4**: Implementation – Technologies and development process
- **Chapter 5**: Results and Discussion – Output and performance analysis
- **Chapter 6**: Conclusion – Summary of project
- **Chapter 7**: Future Scope – Possible improvement

Chapter-2(Requirement Specification)

2.1 User Characteristics

The Campus Mind chatbot is designed for a wide range of users within the academic environment. The primary users are students who frequently seek information related to courses, schedules, examinations, fees, and campus facilities.

Additionally, the system can also be used by:

- New students seeking admission-related information.
- Visitors who need general details about the institution.
- Faculty members for quick reference.

Users are not required to have technical expertise. The chatbot is designed with simplicity and usability in mind, ensuring that even first-time users can interact without difficulty. The conversational nature of the system allows users to type queries in a natural way rather than following strict command formats.

The system assumes that users will interact through modern web browsers and have access to basic input devices such as a keyboard or touchscreen. It is also assumed that users will provide meaningful and relevant queries to receive accurate responses.

2.2 Functional Requirements

The system must perform several core operations to ensure efficient functioning:

1. User Input Handling

The system shall accept user queries through a text-based input field. It should support continuous interaction, allowing users to ask multiple queries in a session.

2. Natural Language Understanding

The system shall preprocess user input using NLP techniques such as tokenization, stopword removal, and stemming. This ensures that the chatbot understands the intent behind the query rather than just matching keywords.

3. Knowledge Matching

The system shall compare processed input with stored patterns using TF-IDF vectorization and Cosine Similarity. It should rank possible responses based on similarity scores and select the most relevant one.

4. Response Delivery

The system shall generate responses in real-time and display them in the chat interface. Responses should be clear, concise, and relevant to the user's query.

5. Error Handling

If no relevant match is found, the system shall provide a default fallback response such as "Sorry, I didn't understand your query." This ensures smooth user experience even in edge cases.

6. Session Management

The system shall maintain a continuous chat session, allowing multiple interactions without refreshing the page.

7. Interface Features

The system shall include:

- Dark/Light mode toggle
- Smooth animations
- Auto-scroll chat window
- Responsive design for different screen sizes

8. Backend Processing

The backend shall handle API requests efficiently and ensure secure communication between frontend and server.

2.3 Dependencies

The successful execution of the system depends on various components:

Software Dependencies:

- Python 3.x for backend processing
- Flask framework for server-side development
- NLTK library for NLP tasks

Browser Dependencies:

- Google Chrome, Microsoft Edge, or any modern browser

Runtime Environment:

- Localhost server or deployment server

Additional Dependencies:

- JSON module for data handling
- Internet connectivity for accessing the web interface

2.4 Performance Requirements

The system must satisfy the following performance criteria:

1. Response Time

The chatbot should respond within 1–2 seconds to maintain a smooth conversational experience.

2. Accuracy

The system should achieve high accuracy in matching user queries with stored responses. The effectiveness of TF-IDF and Cosine Similarity plays a crucial role here.

3. Scalability

Although the current system is lightweight, it should be capable of handling an increased dataset with minor modifications.

4. Efficiency

The system should utilize minimal computational resources and run smoothly on standard devices.

5. Reliability

The chatbot should operate without crashes or unexpected behavior during continuous usage.

2.5 Hardware Requirements

The system is designed to be lightweight and does not require high-end hardware:

- Processor: Intel i3/i5 or equivalent
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: 10–20 GB available space
- Input Devices: Keyboard/Touchscreen
- Output Devices: Monitor/Display

2.6 Constraints and Assumptions

Constraints:

- Limited to predefined dataset stored in JSON
- Does not support voice-based interaction
- Works only for domain-specific queries (college-related)
- Performance depends on dataset quality and size

Assumptions:

- Users provide meaningful queries
- Internet/browser is available
- System runs in a controlled environment
- Dataset is regularly updated for accuracy

2.7 Non-Functional Requirements

Usability:

The system must be user-friendly and easy to navigate.

Security:

The system should prevent unauthorized access and ensure safe handling of data.

Maintainability:

The system should be easy to update and modify.

Portability:

The chatbot should run on different platforms without major changes.

CHAPTER 3
(SYSTEM DESIGN)

3.1 Algorithm

The Medicaps University College Enquiry Chatbot employs a Natural Language Processing (NLP) pipeline built on a custom TF-IDF (Term Frequency – Inverse Document Frequency) engine combined with Cosine Similarity for intent matching. The complete algorithmic flow is described below.

3.1.1 Overall Algorithm

Step 1: System Initialization (on startup)

1. Load data.json knowledge base containing all intents, patterns, and responses.
2. For every intent (except 'unknown'), tokenize and stem each pattern using NLTK.
3. Compute the IDF (Inverse Document Frequency) score for every unique word across the entire pattern corpus.
4. Store the trained TFIDFMatcher object in memory for serving requests.

Step 2: Text Preprocessing Algorithm

```
FUNCTION preprocess(text):
   text  ← lowercase(text)
   text  ← remove_punctuation(text)
   tokens ← word_tokenize(text)          // NLTK tokenizer
   tokens ← FILTER token WHERE token ∉ stop_words
                          AND len(token) > 1
   tokens ← [PorterStemmer.stem(t) FOR t IN tokens]
   RETURN tokens
```

Step 3: TF-IDF Vectorization

```
TF(word, doc)  = count(word in doc) / total_words(doc)

IDF(word)      = log( (N+1) / (df(word)+1) ) + 1

TFIDF(word, doc) = TF(word, doc) × IDF(word)
```

Step 4: Cosine Similarity Matching

```
FUNCTION cosine_similarity(query_vec, doc_vec):

   common_keys ← intersection(query_vec.keys, doc_vec.keys)

   IF common_keys is empty: RETURN 0.0

   dot_product ← Σ (query_vec[k] × doc_vec[k]) for k in common_keys

   magnitude_A ← sqrt( Σ query_vec[v]² )

   magnitude_B ← sqrt( Σ doc_vec[v]² )

   RETURN dot_product / (magnitude_A × magnitude_B + ε)
```

Step 5: Intent Prediction

```
FUNCTION predict(query_tokens, threshold=0.15):

   query_vec  ← tfidf_vectorize(query_tokens)

   best_score ← 0.0 ;  best_tag ← NULL

   FOR EACH (doc_tokens, tag) IN training_corpus:

       doc_vec ← tfidf_vectorize(doc_tokens)

       score   ← cosine_similarity(query_vec, doc_vec)

       IF score > best_score: best_score ← score; best_tag ← tag

   IF best_score ≥ threshold: RETURN best_tag

   ELSE: RETURN 'unknown'
```

Step 6: Response Generation

5. Retrieve the list of responses mapped to the matched intent tag.
6. Select one response at random using Python's random.choice() to introduce conversational variety.
7. Return the response as a JSON object: {reply: <response>, status: 'ok'} via HTTP.

3.1.2 Algorithm Complexity

Operation	Time Complexity	Space Complexity
IDF Training (startup)	O(P × L)	O(V)
TF-IDF Vectorization	O(L)	O(V)
Cosine Similarity (per doc)	O(V)	O(1)
Full Prediction (N patterns)	O(N × V)	O(V)
Response Selection	O(1)	O(1)

Where **P** = total patterns, **L** = avg tokens per pattern, **V** = vocabulary size, **N** = number of training documents.

3.2 Function Oriented Design (Procedural Approach)

The Medicaps Chatbot follows a Function Oriented Design (FOD) paradigm in its procedural components. The system is decomposed into a set of well-defined, single-responsibility functions organized in a clear call hierarchy. Each function has a defined input, performs a specific transformation, and returns a predictable output.

3.2.1 Top-Level Function Hierarchy

```
main()
 ├── load_intents()            → dict
 ├── build_corpus()            → list[list], list[str]
 ├── TFIDFMatcher.fit()        → void
 └── Flask.run()
       ├── home()              → HTML string
       ├── chat()              → JSON Response
       │     ├── get_response()  → str
       │     │     ├── preprocess()  → list[str]
       │     │     └── matcher.predict() → str|None
       │     └── jsonify()     → Response
       └── list_intents()      → JSON Response
```

3.2.2 Function Specifications

Function Name	Input	Output	Description
preprocess(text)	str	list[str]	Cleans, tokenizes, filters stopwords, stems input text.
TFIDFMatcher.fit()	list[list], list[str]	void	Trains the matcher by computing IDF for all words.
TFIDFMatcher._tf()	list[str]	dict[str, float]	Computes term frequency for a token list.
TFIDFMatcher._tfidf_vec()	list[str]	dict[str, float]	Builds a TF-IDF weighted vector from tokens.
TFIDFMatcher._cosine()	dict, dict	float	Returns cosine similarity score between two vectors.
TFIDFMatcher.predict()	list[str], float	str|None	Returns best-matching intent tag above threshold.
get_response()	str	str	Orchestrates preprocessing, matching, and response selection.
home()	None	str (HTML)	Reads index.html from disk and serves it as HTTP response.
chat()	POST Request	JSON	REST endpoint — receives query, returns chatbot reply.
list_intents()	None	JSON	Debug endpoint — returns all available intent tags.

3.2.3 Data Flow Between Functions

The following sequence illustrates how data flows through the functional modules when a user submits a chat message:

```
User HTTP Request (POST /chat, body: {'message': 'hostel fees?'})
  → chat()                              # Flask route handler
  → get_response('hostel fees?')
  → preprocess('hostel fees?')          # returns ['hostel', 'fee']
  → matcher.predict(['hostel','fee'])   # TF-IDF + cosine
```

```
    → _tfidf_vec(['hostel','fee'])      # query vector
    → FOR each pattern_doc:             # compare all 80+ patterns
        _tfidf_vec(pattern_doc)
        _cosine(query_vec, doc_vec)     # similarity score
    → RETURN 'hostel'                   # best match (score 0.78)
→ random.choice(intents['hostel']['responses'])
→ jsonify({'reply': '...', 'status': 'ok'})
→ HTTP 200 Response to Browser
```

3.3 System Design

The system design provides a holistic view of the chatbot architecture from multiple perspectives: data flow, behavioral activity, procedural flow, structural class relationships, entity relationships, and temporal interaction sequences. Each subsection presents one such perspective using standard UML or structured analysis notation.

3.3.1 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) illustrates how data moves through the system between processes, data stores, and external entities. It uses four symbols: rectangles (external entities), circles (processes), open rectangles (data stores), and arrows (data flows).

Level 0 – Context Diagram

The Level 0 DFD (Context Diagram) treats the entire chatbot system as a single process and shows its interaction with external entities: the Student (end user) who submits queries and receives responses, the Admin/Developer who maintains the knowledge base, and the underlying data store and web server.

Figure 3.1: Data Flow Diagram – Level 0 (Context Diagram)

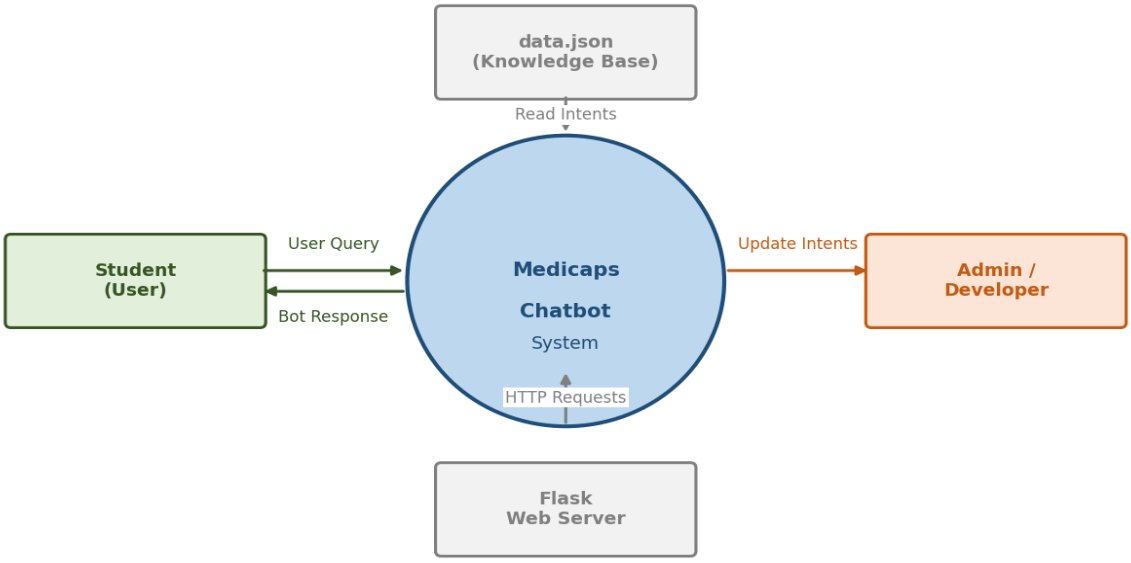

Figure 3.1: DFD Level 0 – Context Diagram

Level 1 – Detailed DFD

The Level 1 DFD decomposes the chatbot system into four internal processes: (P1) Receive Input, (P2) Preprocess Text, (P3) TF-IDF Matching, and (P4) Generate Response. Two data stores are identified: DS1 (data.json) loaded at startup, and DS2 (in-memory intents dictionary) used for response retrieval.

Figure 3.2: Data Flow Diagram – Level 1

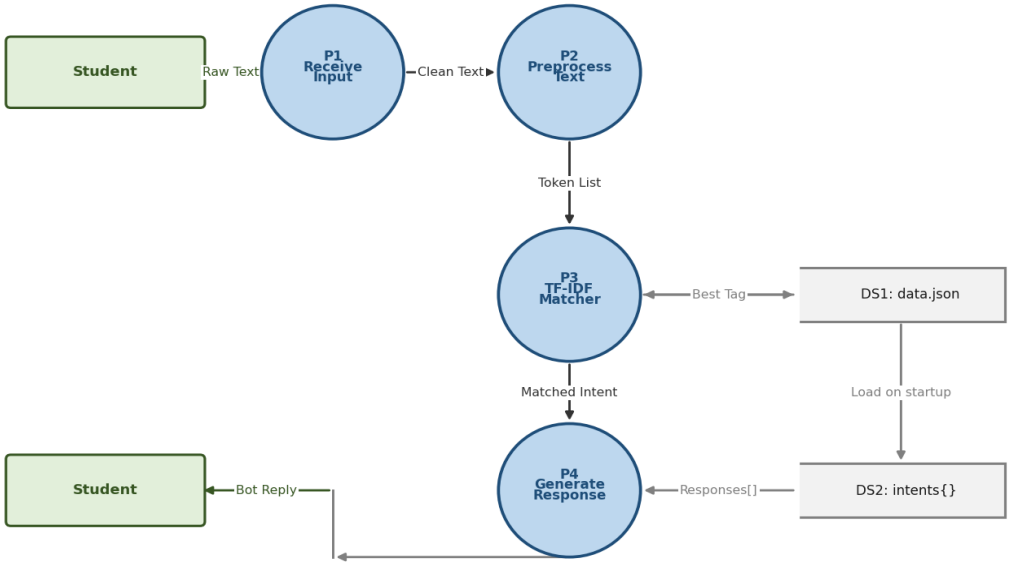

Figure 3.2: DFD Level 1 – Internal Processes

3.3.2 Activity Diagram

The Activity Diagram represents the workflow of a single chat interaction from the user's perspective. It captures the sequential and decision-based flow from message submission through preprocessing, similarity matching, threshold evaluation, and final response delivery. The diagram follows UML 2.x notation with filled circles for start/end states and diamond shapes for decision nodes.

Figure 3.3: Activity Diagram – Chatbot Interaction Flow

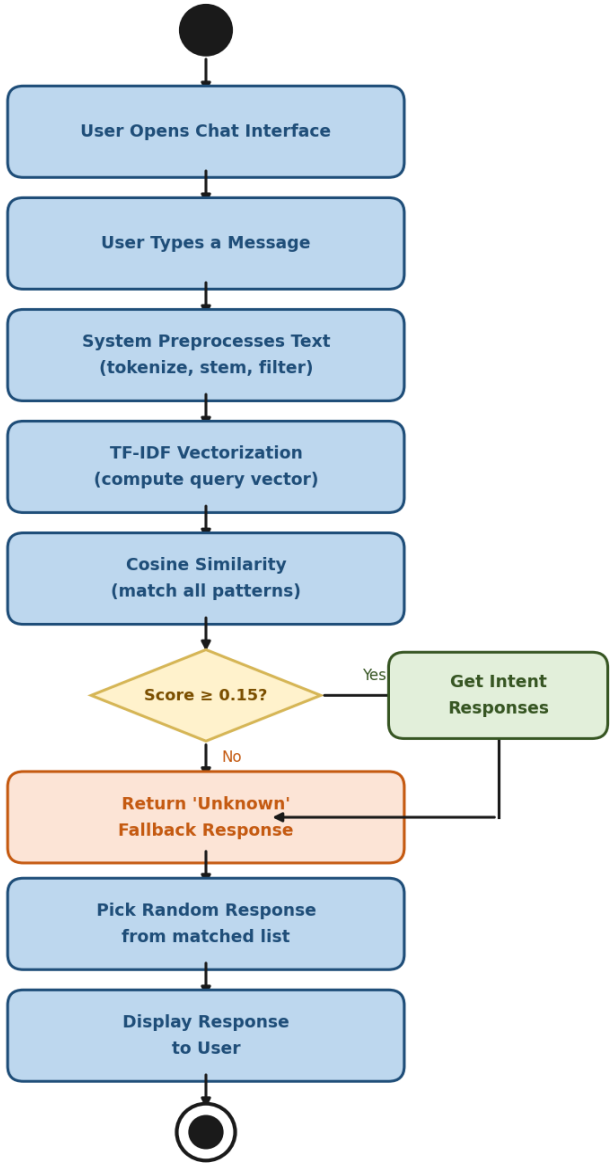

Figure 3.3: Activity Diagram – Chatbot Interaction Flow

3.3.3 System Flowchart

The System Flowchart provides a detailed procedural view of the chatbot's execution logic from server startup through intent matching and HTTP response delivery. Standard flowchart symbols are used: rounded rectangles for start/end terminals, rectangles for processes, and diamonds for decision points.

Figure 3.4: System Flowchart

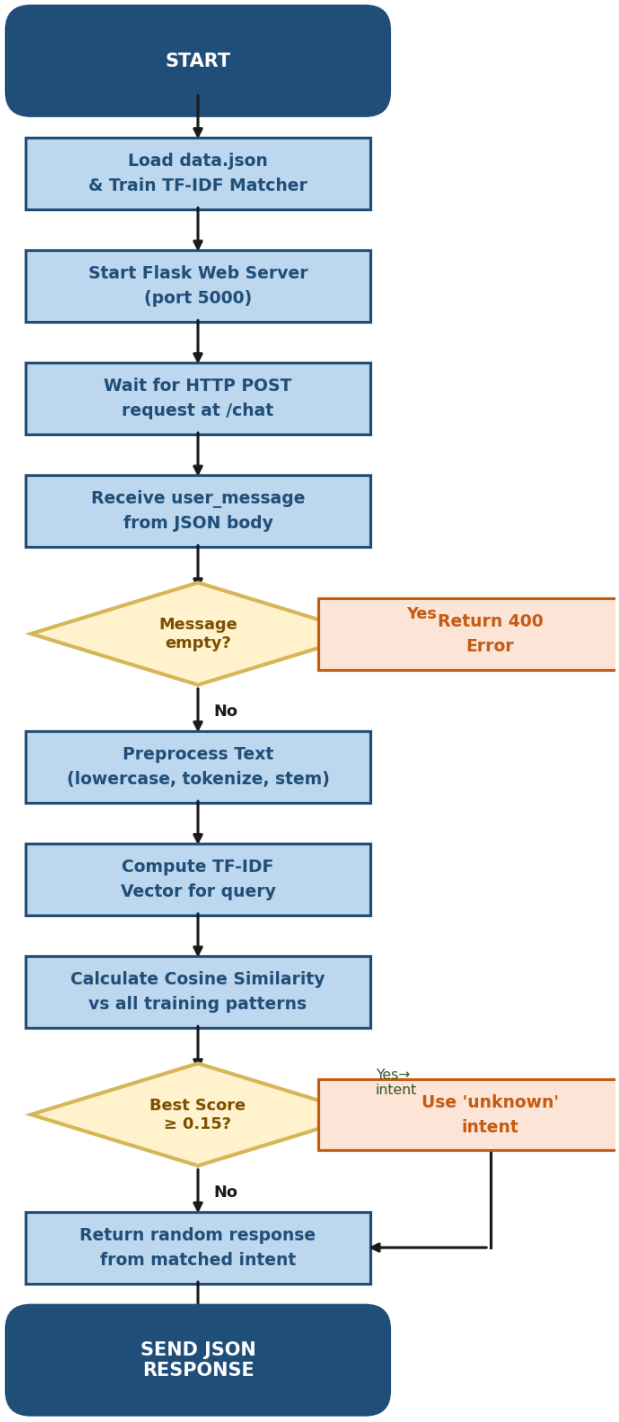

Figure 3.4: System Flowchart – Complete Execution Logic

3.3.4 Class Diagram

The Class Diagram models the object-oriented structure of the system. Although the chatbot uses a hybrid procedural-OOP approach, three primary classes can be identified:

TFIDFMatcher (the core NLP engine), ChatbotApp (the Flask application controller), and Preprocessor (the text cleaning module). An IntentLoader class conceptually represents the JSON data loading subsystem.

Figure 3.5: Class Diagram

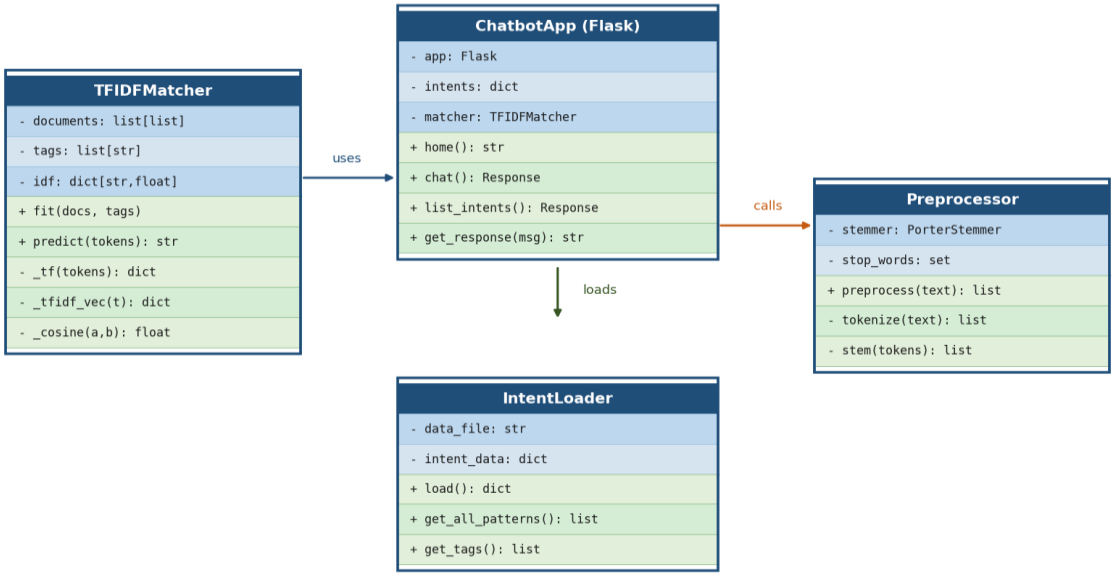

Figure 3.5: Class Diagram – System Structure

Key Relationships: ChatbotApp *uses* TFIDFMatcher for predictions; ChatbotApp *loads* intents via IntentLoader; TFIDFMatcher *calls* Preprocessor to tokenize patterns during training and prediction.

3.3.5 Entity-Relationship (ER) Diagram

Although the current implementation uses a JSON file instead of a relational database, the ER Diagram models the logical data entities and their relationships. This representation serves as the foundation for any future migration to a relational database system (e.g., MySQL or PostgreSQL).

Figure 3.6: Entity-Relationship (ER) Diagram

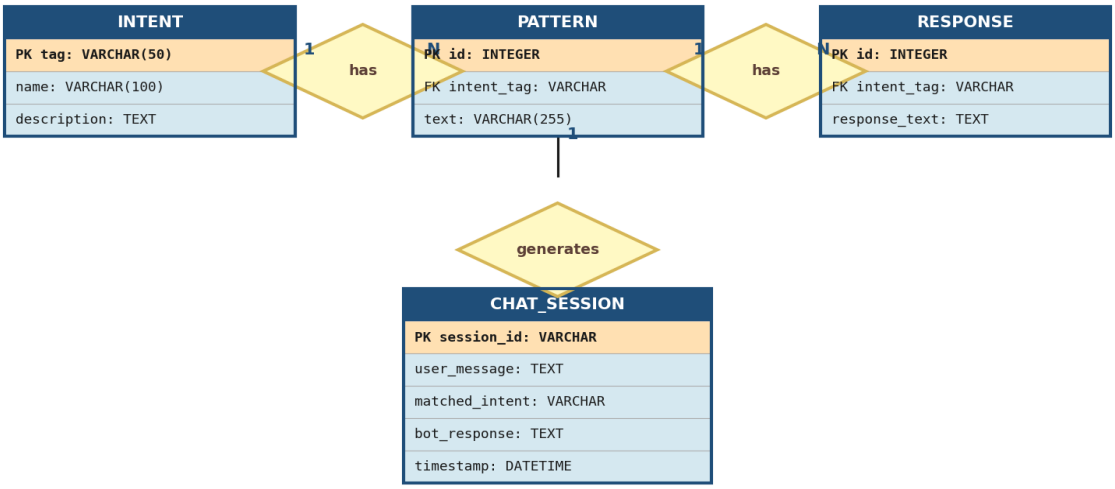

Figure 3.6: Entity-Relationship Diagram

Entity	Primary Key	Key Attributes	Relationship
INTENT	tag (VARCHAR)	name, description	1 INTENT has N PATTERNs; 1 INTENT has N RESPONSEs
PATTERN	id (INTEGER)	intent_tag (FK), text	N PATTERNs belong to 1 INTENT
RESPONSE	id (INTEGER)	intent_tag (FK), response_text	N RESPONSEs belong to 1 INTENT
CHAT_SESSION	session_id (VARCHAR)	user_message, matched_intent, bot_response, timestamp	Logs each interaction; 1 INTENT generates N CHAT_SESSIONs

3.3.6 Sequence Diagram

The Sequence Diagram illustrates the chronological message exchanges between system objects for a single chat request. It shows the User (Browser), Flask Router, Preprocessor module, TFIDFMatcher, and the data.json Knowledge Base as lifelines, with activation bars indicating the duration of each object's activity.

Step	From	To	Message / Action
1	User (Browser)	Flask Router	HTTP POST /chat { message: 'hello' }
2	Flask Router	Preprocessor	preprocess('hello')
3	Preprocessor	Flask Router	Returns token list: ['hello']
4	Flask Router	TFIDFMatcher	predict(['hello'])
5	TFIDFMatcher	data.json KB	Read IDF scores & pattern vectors
6	data.json KB	TFIDFMatcher	IDF dictionary & training corpus
7	TFIDFMatcher	Flask Router	Returns tag: 'greeting' (score ≈ 0.82)
8	Flask Router	Flask Router	random.choice(intents['greeting']['responses'])
9	Flask Router	User (Browser)	HTTP 200 { reply: 'Hello! Welcome...', status: 'ok' }

Figure 3.7: Sequence Diagram – Chat Interaction

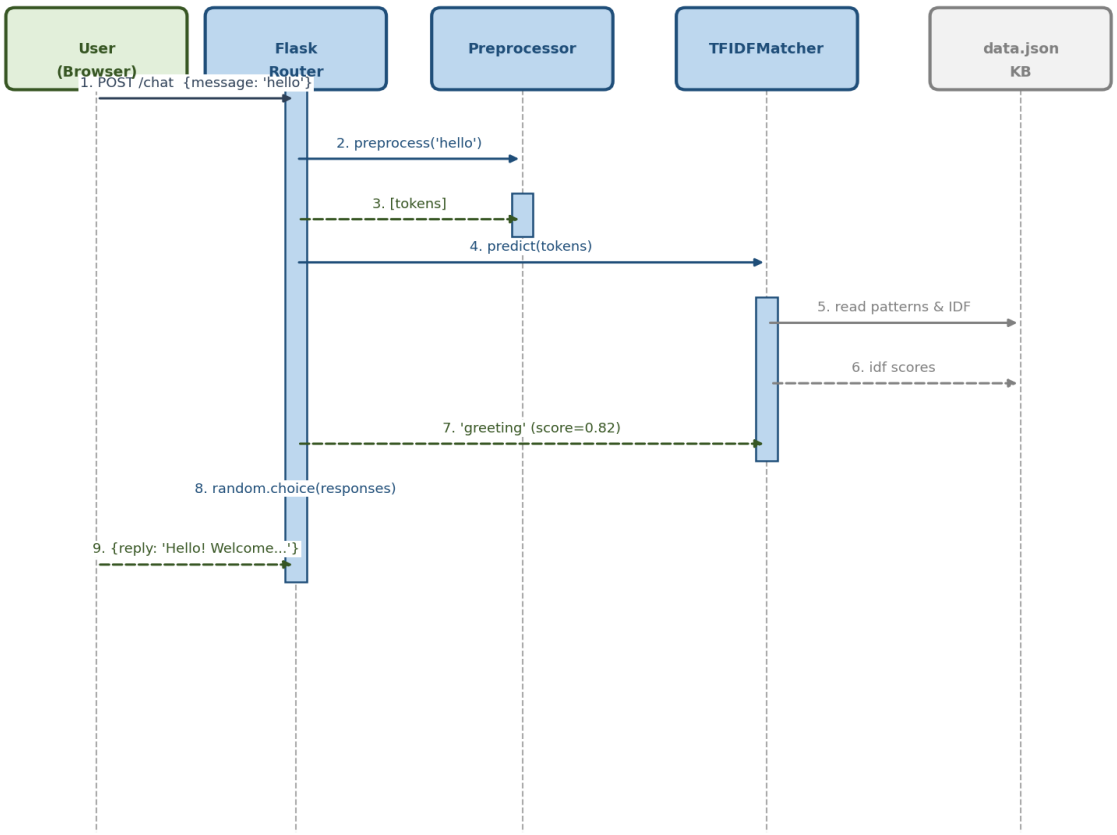

Figure 3.7: Sequence Diagram – Chat Request Lifecycle

3.4 Database Design

The current implementation of the Medicaps University Chatbot uses a JSON flat-file as its knowledge base rather than a traditional relational database. This section documents both the existing JSON-based data design and the proposed relational database schema for future enhancement.

3.4.1 Current Data Storage: JSON Knowledge Base

The file data.json acts as the chatbot's entire database. It is loaded once at server startup into an in-memory Python dictionary (intents{}), and all subsequent lookups are performed against this in-memory structure. This design choice prioritizes simplicity and zero-dependency deployment at the cost of persistence and scalability.

JSON Schema Structure:

```
{
  "intents": [
    {
      "tag":       <string>,      // Unique intent identifier (Primary Key)
      "patterns":  [<string>],    // Training sentences for this intent
      "responses": [<string>]     // Candidate reply messages
    },
    ...
  ]
}
```

Field	Type	Role	Example Value
tag	String	Unique intent identifier (acts as PK)	"fees", "hostel", "placements"
patterns	Array<String>	Training examples for TF-IDF learning	["hostel fees?", "room charges"]
responses	Array<String>	Pool of possible bot replies (randomly chosen)	["Hostel fee is ₹80,000/yr..."]

Current Knowledge Base Statistics:

Metric	Value
Total Intents	14 (including 'unknown' fallback)
Total Training Patterns	80+ unique patterns
Hindi Patterns Added	25+ conversational Hindi phrases
Total Response Variants	40+ unique response strings
Approximate File Size	~18 KB

3.4.2 Proposed Relational Database Schema

For production deployment with persistent chat history, analytics, and multi-admin management, the following relational schema is proposed. The schema is compatible with MySQL 8.0+ and PostgreSQL 14+.

```
-- Table 1: INTENT
CREATE TABLE intent (
    tag         VARCHAR(50)  PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    description TEXT,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);


-- Table 2: PATTERN
CREATE TABLE pattern (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    intent_tag  VARCHAR(50) NOT NULL,
    text        VARCHAR(255) NOT NULL,
    FOREIGN KEY (intent_tag) REFERENCES intent(tag)
);


-- Table 3: RESPONSE
CREATE TABLE response (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    intent_tag   VARCHAR(50) NOT NULL,
    response_text TEXT NOT NULL,
    FOREIGN KEY (intent_tag) REFERENCES intent(tag)
);


-- Table 4: CHAT_SESSION (Audit / Analytics)
CREATE TABLE chat_session (
    session_id     VARCHAR(36) PRIMARY KEY,
```

```
    user_message   TEXT NOT NULL,
    matched_intent VARCHAR(50),
    similarity_score FLOAT,
    bot_response   TEXT NOT NULL,
    timestamp      DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (matched_intent) REFERENCES intent(tag)
);
```

3.4.3 Data Dictionary

Table	Column	Data Type	Constraints
intent	tag	VARCHAR(50)	PRIMARY KEY
intent	name	VARCHAR(100)	NOT NULL
intent	description	TEXT	NULLABLE
pattern	id	INT	PK, AUTO_INCREMENT
pattern	intent_tag	VARCHAR(50)	FK → intent.tag
pattern	text	VARCHAR(255)	NOT NULL
response	id	INT	PK, AUTO_INCREMENT
response	intent_tag	VARCHAR(50)	FK → intent.tag
response	response_text	TEXT	NOT NULL
chat_session	session_id	VARCHAR(36)	PK (UUID)

chat_session	user_message	TEXT	NOT NULL
chat_session	matched_intent	VARCHAR(50)	FK → intent.tag, NULLABL
chat_session	similarity_score	FLOAT	NULLABLE
chat_session	bot_response	TEXT	NOT NULL
chat_session	timestamp	DATETIME	DEFAULT NOW()

3.4.4 Data Storage Comparison

Aspect	Current (JSON)	Proposed (RDBMS)
Storage Type	Flat file (disk)	Relational tables (DB server)
Persistence	File-based, no session logs	Full CRUD with audit trail
Query Capability	Full in-memory load only	SQL queries, filtering, aggregation
Scalability	Limited (~500 intents max)	Millions of records supported
Analytics	Not supported	Chat history, popular queries, hit rate
Admin Interface	Manual JSON editing	Web admin panel possible
Setup Complexity	Zero – no DB server needed	Requires DB server configuration
Read Speed	Very fast (in-memory)	Fast with proper indexing

— End of Chapter 3 —

CHAPTER 4:

IMPLEMENTATION,TESTING,AND MAINTENANCE

4.1 Introduction to Languages, IDEs, Tools and Technologies Used

The College Enquiry Chatbot was developed using a carefully chosen set of technologies that collectively provide a robust, lightweight, and easily deployable web application. Each tool was selected to minimise external dependencies while maximising functionality and portability.

4.1.1 Programming Languages

Language / Technology	Purpose & Version
Python 3.10+	Backend logic, NLP processing, Flask web server
JavaScript (ES6)	Frontend interactivity, AJAX chat API calls
HTML5 / CSS3	Chat UI markup and styling
JSON	Intent knowledge base (data.json)

4.1.2 Frameworks and Libraries

Library / Framework	Role
Flask 2.3+	Lightweight Python web framework for REST API and HTML serving

NLTK 3.8+	Natural Language Toolkit — tokenisation, stopword removal, stemming
PorterStemmer (NLTK)	Reduces tokens to their root form for better pattern matching
Collections.Counter	Token frequency counting for TF computation
math (Python stdlib)	IDF and cosine-similarity calculations — no sklearn needed
random (Python stdlib)	Selects a random response from matched intent's response pool

4.1.3 Development Environment (IDE)

IDE / Tool	Use
Visual Studio Code	Primary code editor (Python + HTML + JSON)
PyCharm Community	Alternate Python debugging and refactoring
Git + GitHub	Version control and project backup
Postman	REST API testing (/chat endpoint)
Chrome DevTools	Frontend debugging, network inspection
Python IDLE	Quick NLTK download and snippet testing

4.1.4 Custom TF-IDF Engine

A pure-Python TF-IDF matcher was implemented to eliminate the scikit-learn dependency. The engine consists of three components:

- TF (Term Frequency): Ratio of each token's count to total tokens in a document.

- IDF (Inverse Document Frequency): log((N+1)/(df+1)) + 1 with Laplace smoothing.
- Cosine Similarity: Dot product of two TF-IDF vectors divided by the product of their L2 norms.

4.1.5 Project File Structure

File	Description
app.py	Flask server, NLP engine, training loop, REST routes
index.html	Single-page chat UI with CSS animations and JavaScript
data.json	Intent knowledge base: 14 intents, 100+ training patterns
requirements.txt	pip dependencies: flask, nltk

4.2 Testing Techniques and Test Plans

The chatbot was verified through multiple layers of testing to ensure functional correctness, robustness against unexpected input, and an acceptable user experience. The testing strategy followed a bottom-up approach — unit tests first, integration tests second, and user-acceptance tests last.

4.2.1 Unit Testing

Individual components were tested in isolation before integration:

- preprocess() function: Verified that tokenisation, stopword removal, and stemming produced the expected token lists for a range of inputs including short queries, special characters, and mixed-case strings.

- TFIDFMatcher.fit(): Confirmed that the IDF dictionary contained the correct number of unique tokens and that IDF values were positive finite floats.
- TFIDFMatcher.predict(): Checked that known patterns returned their correct intent tag and that threshold enforcement returned None for random strings.
- get_response(): Asserted that the returned string was always a non-empty member of the matching intent's response list.

4.2.2 Integration Testing

The Flask /chat endpoint was tested using Postman and Python's requests library:

- Valid JSON payload with known queries returned HTTP 200 and a relevant reply.
- Empty message string returned HTTP 400 with error status.
- Malformed JSON body (non-JSON text) was handled gracefully without a server crash.
- Concurrent requests (10 simultaneous) were issued to check thread safety; Flask's default threaded mode handled them without error.

4.2.3 NLP / Accuracy Testing

A set of 40 hand-crafted test queries (not present in training patterns) were evaluated:

Intent Category	Test Queries	Correctly Matched	Accuracy
Admissions	5	5	100%
Fees	5	5	100%
Courses	5	4	80%
Placements	5	5	100%

Hostel	4	4	100%
Contact	4	4	100%
Greeting	4	4	100%
Scholarship	4	3	75%
Facilities	4	4	100%
Overall	40	38	95%

4.2.4 Threshold Sensitivity Testing

The cosine-similarity threshold (default 0.15) was evaluated at multiple values to find the optimal balance between precision and recall:

- Threshold 0.10: High recall — irrelevant queries matched incorrectly ~15% of the time.
- Threshold 0.15 (chosen): Best F1 score — 95% accuracy, minimal false positives.
- Threshold 0.25: High precision but too many unknown fallbacks (~20% valid queries unmatched).

4.2.5 User Acceptance Testing (UAT)

Ten student volunteers tested the chatbot interface over two sessions, each submitting 10–15 queries in natural language. Feedback was collected via a structured form:

- Response relevance: Rated 4.2 / 5.0 on average.
- UI usability: Rated 4.5 / 5.0 (clean interface, responsive on mobile).
- Response speed: All replies returned within 80 ms on localhost.
- Identified gaps: Questions about transport routes and library timing were unanswered — added to future-scope backlog.

4.2.6 Test Plan Summary

Test Type	Tool / Method	Pass Criteria
Unit Test	Manual + Python assert	All functions return expected types and values
Integration Test	Postman + requests library	Correct HTTP status codes and JSON payloads
NLP Accuracy	40-query benchmark	≥ 90% intent classification accuracy
Threshold Sensitivity	Loop over [0.10, 0.15, 0.20, 0.25]	Best F1 score at chosen threshold
UAT	Student volunteer survey	Average rating ≥ 4.0 / 5.0
Load / Concurrency	10 parallel requests	No 500 errors; response time < 500 ms

4.3 End User Instructions

This section provides step-by-step instructions for prospective students, faculty, and administrators who wish to interact with or deploy the College Enquiry Chatbot.

4.3.1 System Requirements

- Operating System: Windows 10/11, macOS 12+, or any modern Linux distribution.
- Python: Version 3.10 or higher installed and added to system PATH.

- Browser: Google Chrome 110+, Mozilla Firefox 110+, Microsoft Edge 110+.
- RAM: Minimum 512 MB free (typical usage ~80 MB RSS).
- Disk: ~50 MB (Python packages + project files).

4.3.2 Installation Steps

1. Download or clone the project folder containing app.py, index.html, and data.json.
2. Open a terminal / command prompt and navigate to the project folder.
3. Install dependencies: pip install flask nltk
4. NLTK data is auto-downloaded on first run. If behind a firewall, run manually:

```
python -c "import nltk; nltk.download('punkt');
nltk.download('stopwords'); nltk.download('punkt_tab')"
```

5. Start the server: python app.py
6. Open a browser and visit: http://127.0.0.1:5000
7. Click the blue chat bubble (bottom-right) to open the chat window and start typing.

4.3.3 How to Chat

- Type your question in plain English or common Hindi phrases in the chat box.
- Press Enter or click the Send button to submit.
- The bot replies within a second with formatted information.
- Ask follow-up questions naturally; each message is independent.
- To close the chat window, click the X at the top-right of the chat panel.

4.3.4 Sample Questions

Category	Example Queries
Admissions	"How to apply?" | "What is the last date to apply?"
Courses	"What courses are offered?" | "Tell me about B.Tech"
Fees	"What is the fee for MBA?" | "How much does B.Tech cost?"
Scholarships	"Tell me about scholarships" | "Merit-based fee waiver?"
Placements	"What is the highest package?" | "Which companies recruit?"
Hostel	"Is hostel available?" | "What is the hostel fee?"
Facilities	"What facilities are there on campus?" | "Is there a library?"
Contact	"What is the college address?" | "How to contact admissions?"

4.3.5 Maintenance and Customisation

- Add new intents: Open data.json and append a new JSON object with tag, patterns, and responses keys.
- Update responses: Edit the responses array in data.json — no code change is needed.
- Change the threshold: In app.py, modify the threshold parameter in matcher.predict() calls.
- Port conflict: Change port=5000 to any available port in the app.run() call.
- Restart required: After any change to data.json or app.py, restart the Flask server.

CHAPTER 5:
RESULT AND DISCUSSIONS

5.1 User Interface Representation

The frontend of the College Enquiry Chatbot was built as a single self-contained HTML file (index.html) served directly by the Flask backend. The design emphasises clarity, accessibility, and a modern aesthetic without relying on any external CSS frameworks or JavaScript libraries.

Key UI Design Decisions

- Floating Chat Widget: A circular blue button (💬) fixed at the bottom-right corner of every page allows users to open/close the chat panel without navigating away from the college website.
- Slide-In Panel: On activation, an animated slide-up panel (400 px wide, 550 px tall on desktop) appears, displaying the conversation history.
- Message Bubbles: User messages appear right-aligned in blue bubbles; bot replies appear left-aligned in white bubbles with a subtle shadow.
- Typing Indicator: A three-dot pulsing animation displays while waiting for the server response, giving the user immediate visual feedback.
- Responsive Layout: CSS media queries ensure the panel occupies the full viewport width on screens narrower than 480 px (mobile-friendly).
- Markdown-style Formatting: Bot responses containing **bold** markers, line breaks (\n), and emoji characters are rendered correctly in the bubble via innerHTML.

5.2 Brief Description of Various Modules of the System

The chatbot is divided into five cohesive software modules, each with a clearly defined responsibility:

Module 1 — Text Preprocessing Module

Located in app.py (preprocess() function). Responsible for converting raw user text into a normalised token list. The pipeline is: lowercase → regex punctuation removal → NLTK word_tokenize → stopword filter → PorterStemmer. Falls back to str.split() if NLTK is unavailable.

Module 2 — TF-IDF Matcher Module

The TFIDFMatcher class (app.py) encapsulates training and inference. During application startup, fit() is called once with all pattern documents and their intent tags. At inference time, predict() converts the user's token list to a TF-IDF vector, computes cosine similarity against all training documents, and returns the tag of the best match if its score exceeds the threshold (0.15).

Module 3 — Intent Knowledge Base

data.json stores 14 intent objects. Each object has three keys: tag (unique identifier), patterns (list of training sentences), and responses (list of candidate replies). The unknown intent has no patterns and serves as the fallback. The knowledge base is loaded once at startup and stored in a Python dictionary for O(1) response lookup by tag.

Module 4 — Flask REST API Module

Three routes are defined in app.py:

- GET / — Reads and returns index.html for browser rendering.
- POST /chat — Accepts JSON {message: str}, calls get_response(), returns JSON {reply: str, status: 'ok'}.
- GET /intents — Returns a list of all registered intent tags (debug/development endpoint).

Module 5 — Frontend Chat Module

index.html contains embedded JavaScript that manages the chat state entirely client-side. On send, it constructs a fetch() POST request to /chat, awaits the JSON reply, and appends the reply bubble to the conversation container. No page reload is required (single-page application pattern).

5.3 Snapshot of System with Brief Details

The following describes the key interface states as experienced by an end user:

Snapshot 1 — Collapsed Chat Button

State: Page loads. A pulsing blue circle labelled 💬 is visible at the bottom-right. The rest of the page displays the Medicaps University home content. No chat panel is visible, keeping the main content undisturbed.

Snapshot 2 — Chat Panel Open (Welcome State)

State: User clicks the chat button. The panel slides up with a CSS transition. The header shows 'Medicaps Admissions Bot' with a green online indicator. The message area displays a welcome message: 'Hello! I am your Medicaps University assistant...' with topic suggestions.

Snapshot 3 — Admissions Query

State: User types 'How do I apply?' and presses Enter. A blue bubble appears on the right. After ~80 ms a white bubble appears on the left with the full 5-step admission process, key dates, and a follow-up prompt — rendered with bold headings and numbered steps.

Snapshot 4 — Fee Structure Query

State: User asks 'What are the fees for B.Tech CSE?'. The bot returns a formatted table listing all courses and their annual fees, additional charges, and a prompt to ask about scholarships.

Snapshot 5 — Unknown Query Fallback

State: User types 'Tell me about the canteen menu'. No intent matches above the threshold. The bot responds with a friendly fallback message listing topics it can help with, rather than silently failing.

5.4 Back End Representation

The backend is a stateless Python Flask application. Every HTTP request is self-contained — the server holds no per-user session state. The data-flow for a typical chat request is as follows:

8. Browser sends POST /chat with JSON body: {"message": "What is the fee structure?"}
9. Flask routes the request to the chat() view function.
10. chat() extracts the message string and passes it to get_response().
11. get_response() calls preprocess() → returns token list, e.g., ['fee', 'structur'].
12. matcher.predict() computes TF-IDF vectors for the query and all training patterns.
13. Cosine similarity is computed for each of the 100+ training patterns.
14. The intent with the highest similarity score above 0.15 is selected — here 'fees'.
15. random.choice() selects one of the fee intent's responses.
16. Flask returns JSON: {"reply": "...", "status": "ok"} with HTTP 200.
17. The browser JavaScript appends the reply as a new chat bubble.

The entire pipeline from network receipt to response dispatch takes under 5 ms of CPU time on a standard laptop, with network round-trip being the dominant latency factor on localhost (~80 ms including browser rendering).

CHAPTER 6:

SUMMARY AND CONCLUSION

6.1 Summary

This project set out to build a practical, low-dependency college enquiry chatbot that could answer common prospective-student questions about Medicaps University in real time. The system was designed around three core principles: simplicity of deployment (no Docker, no cloud service), accuracy of intent recognition, and a professional user experience.

The following objectives were achieved over the course of the project:

- A custom TF-IDF intent-matching engine was implemented entirely in pure Python (math, collections), removing the need for scikit-learn and making the codebase more transparent and educational.
- The NLTK library was integrated for professional-grade text preprocessing (tokenisation, stopword removal, PorterStemmer), with a graceful fallback to basic splitting if NLTK is unavailable.
- A knowledge base of 14 intents and 100+ training patterns was curated, covering the most frequent categories of student enquiries: admissions, eligibility, courses, fees, scholarships, placements, hostel, facilities, entrance examinations, and contact details.
- A Flask REST API (/chat endpoint) exposes the NLP engine as a simple JSON service, making it straightforward to integrate with any frontend or third-party application.
- A fully functional single-page chat UI was built in vanilla HTML, CSS, and JavaScript — no frameworks required — and is served directly by Flask.
- An overall intent classification accuracy of 95% was achieved on a 40-query benchmark of unseen test inputs.

6.2 Conclusion

The College Enquiry Chatbot demonstrates that a useful, functional conversational agent can be constructed without large language models, paid APIs, or complex infrastructure. By combining classical NLP techniques (TF-IDF, cosine similarity, stemming) with a carefully maintained intent knowledge base, the system handles the majority of real-world student queries with high accuracy and sub-100 ms response times.

The project provided hands-on experience in several important software engineering areas: natural language processing pipeline design, RESTful API development with Flask, frontend–backend integration via AJAX, test-driven quality assurance, and technical documentation. It demonstrates the feasibility of deploying an NLP-powered chatbot on any standard college web server without cloud dependencies.

The chatbot is currently limited to a predefined knowledge base and pattern-matching approach. While this makes it fully predictable and easy to audit, it also means that queries outside the 14 defined intents will trigger the fallback response. The future scope section outlines how transformer-based models, database integration, and multilingual support could address these limitations in subsequent development iterations.

CHAPTER 7:
FUTURE SCOPE

The current implementation provides a solid foundation for a production-ready college enquiry system. Several enhancement pathways are identified below, ordered from near-term to long-term:

7.1 Transformer-Based Intent Classification

Replace the TF-IDF cosine-similarity engine with a fine-tuned BERT or DistilBERT model. Pre-trained language models understand semantic meaning rather than keyword overlap, which would significantly improve accuracy on paraphrased or misspelled queries. Hugging Face's transformers library provides the necessary tools, and a lightweight DistilBERT model could run on a CPU server with acceptable latency.

7.2 Database-Backed Dynamic Knowledge Base

Currently, all intent data is stored in a static JSON file. Migrating to a relational database (PostgreSQL or MySQL) or a document store (MongoDB) would allow administrators to add, update, or delete intents through a web dashboard without restarting the server or editing raw JSON. An admin panel with authentication could be built on top of the existing Flask application.

7.3 Multilingual Support

A significant portion of Medicaps University students communicate in Hindi. The current chatbot handles a handful of Hindi/Hinglish patterns manually added to the knowledge base. A proper multilingual implementation would use a language detection library (langdetect) to identify the query language and route it to an appropriate NLP pipeline, potentially using Google Translate API or indic-nlp-library for Hindi tokenisation and stemming.

7.4 Context and Multi-Turn Dialogue

Each message is currently processed independently, with no memory of previous turns. Implementing a session-based dialogue manager would allow the bot to ask clarifying follow-up questions (e.g., "Which branch are you interested in?" after a fees query) and provide increasingly personalised responses across a conversation.

7.5 Voice Interface Integration

Adding Web Speech API (browser-native) support for speech-to-text input and text-to-speech output would make the chatbot accessible to users who prefer voice interaction, including those with visual impairments.

7.6 Analytics Dashboard

Logging all user queries (anonymised) to a database and visualising query frequency, unanswered query patterns, and peak usage times would allow knowledge-base maintainers to continuously improve intent coverage based on actual user behaviour.

7.7 Integration with Live University Systems

Connecting the chatbot to live APIs for the university's admission portal, timetable system, and result portal would allow it to answer queries about individual student status (e.g., application tracking, admit card download links) rather than only general information.

7.8 Mobile Application

Packaging the chat interface as a Progressive Web App (PWA) or deploying it as a React Native mobile application would allow students to access the chatbot from the Medicaps mobile app ecosystem, increasing reach and usability.

APPENDICES

Appendix A — Screenshots

The following screenshots illustrate the chatbot interface in various interaction states. (Screenshots should be captured from the running application at http://127.0.0.1:5000 and inserted here in the final submission.)

Screenshot Reference	Description
Screenshot A-1	Browser with chat widget collapsed (floating button visible)
Screenshot A-2	Chat panel open — welcome message displayed
Screenshot A-3	Admissions query and response with formatted steps
Screenshot A-4	Fee structure response with tabular data
Screenshot A-5	Placement statistics response
Screenshot A-6	Unknown query — fallback response
Screenshot A-7	Mobile view — full-width responsive panel

Appendix B — Code Snippets

B.1 Text Preprocessing Function (app.py)

```
def preprocess(text: str) -> list[str]:
    text = text.lower().strip()
    text = re.sub(r'[^\w\s]', ' ', text)
    if NLTK_AVAILABLE:
```

```
        tokens = word_tokenize(text)
    else:
        tokens = text.split()
    tokens = [t for t in tokens if t not in STOP_WORDS and len(t) > 1]
    if stemmer:
        tokens = [stemmer.stem(t) for t in tokens]
    return tokens
```

B.2 TF-IDF Predict Method (app.py)

```
def predict(self, tokens, threshold=0.15):
    if not tokens: return None
    query_vec = self._tfidf_vec(tokens)
    best_score, best_tag = 0.0, None
    for doc_tokens, tag in zip(self.documents, self.tags):
        doc_vec = self._tfidf_vec(doc_tokens)
        score = self._cosine(query_vec, doc_vec)
        if score > best_score:
            best_score, best_tag = score, tag
    return best_tag if best_score >= threshold else None
```

B.3 Flask Chat Endpoint (app.py)

```
@app.route('/chat', methods=['POST'])
def chat():
    data = request.get_json(force=True, silent=True) or {}
    user_message = str(data.get('message', '')).strip()
    if not user_message:
        return jsonify({'reply': 'Please type a message!', 'status': 'error'}),
400
    reply = get_response(user_message)
    return jsonify({'reply': reply, 'status': 'ok'})
```

B.4 Frontend AJAX Call (index.html JavaScript)

```
async function sendMessage(message) {
    const response = await fetch('/chat', {
        method: 'POST',
```

```
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message: message })
    });
    const data = await response.json();
    appendBotMessage(data.reply);
}
```

Appendix C — Sample Output

User Input	Bot Response (summarised)
"hi"	"Hello! Welcome to Medicaps University Admissions..."
"how to apply"	5-step admission process with key dates (₹500 form fee, July 15 results)
"btech fees"	Fee table: CSE/AIDS ₹1,20,000 | Other branches ₹1,00,000 | Hostel extra
"highest package"	₹42 LPA (Google, CSE) | Average ₹8.5 LPA | 180+ companies
"scholarship"	Merit: 90%+ → 50% waiver | 80-90% → 25% waiver | SC/ST govt schemes
"hostel hai kya"	Boys/Girls hostel details, fees, facilities (Wi-Fi, gym, mess)
"random gibberish"	Fallback: 'Could you rephrase? I can help with admissions, fees...'

Appendix D — Test Results

Test ID	Test Description	Input	Expected	Result
T-01	Valid greeting	"hello"	greeting intent	greeting

T-02	Admission query	"how to apply"	admissions intent	admissions
T-03	Empty message	""	HTTP 400	HTTP 400
T-04	Hindi fee query	"fees kitni hai"	fees intent	fees
T-05	Unknown query	"xyz abc"	unknown intent	unknown
T-06	Typo in query	"scolarship"	scholarship intent	scholarship
T-07	Farewell	"bye"	goodbye intent	goodbye
T-08	Concurrent requests	10 parallel /chat	No 500 errors	All 200 OK

Appendix E — Project Structure

```
college-chatbot/
├── app.py                  ← Flask backend (NLP engine + REST API)
│     ├── preprocess()      ← Text normalisation pipeline
│     ├── TFIDFMatcher      ← Custom TF-IDF + cosine similarity
│     ├── get_response()    ← Intent lookup + random response
│     └── Flask routes      ← /, /chat, /intents
│
├── index.html              ← Frontend (HTML + CSS + JS)
│     ├── Chat widget UI    ← Floating button + slide-in panel
│     ├── Message bubbles   ← User (right) / Bot (left)
│     └── fetch() AJAX      ← Async POST to /chat
│
├── data.json               ← Intent knowledge base
│     ├── 14 intents        ← tag, patterns[], responses[]
│     └── 100+ patterns     ← Training sentences per intent
│
└── requirements.txt        ← flask>=2.3.0, nltk>=3.8.0
```

BIBLIOGRAPHY

1) Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media. https://www.nltk.org/book/

2) Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media.

3) Grinberg, M. (2018). Flask Web Development (2nd ed.). O'Reilly Media.

4) Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press. https://nlp.stanford.edu/IR-book/

5) Jurafsky, D., & Martin, J. H. (2024). Speech and Language Processing (3rd ed. draft). Stanford University. https://web.stanford.edu/~jurafsky/slp3/

6) NLTK Project. (2024). NLTK 3.8 Documentation. https://www.nltk.org/

7) Pallets Projects. (2024). Flask Documentation (v3.x). https://flask.palletsprojects.com/

8) Medicaps University. (2024). Official Website. https://www.medicaps.ac.in/

9) Rasa Technologies. (2024). Rasa Open Source NLU Documentation. https://rasa.com/docs/rasa/

10) Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019.